AI-Assisted Reproducibility in Scientific Software

Best Practices for Sustainable AI and HPC Workflows



Made with Napkin

The Reproducibility Challenge

Reproducing results requires more than preserving source code.

- Scientific software combines code, data, ML models, dependencies, and specialized hardware.
- Capture software configuration and dependency tools.
- Record execution environments and evidence supporting results.

AI: Challenge and Opportunity

AI should augment established practices, not replace human judgment.

- Challenges: model versions, parameters, random seeds, rapidly changing libraries.
- Opportunities: documentation, repository inspection, workflow diagnostics.
- Principle: transparent evidence and human scientific judgment remain essential.

Sustainable Software Foundations

Version control and documentation underpin reproducibility.

- Use Git for code, configuration, and documentation; link releases to experiments.
- README.md: explain purpose, installation, execution, and HPC requirements.
- Make code review and automated testing part of routine development.

Dependency and Environment Management

Declare dependencies to prevent unexpected changes.

- Use requirements.txt, pyproject.toml, or environment.yml.
- Record versions when they affect behavior.
- Balance loose constraints (drift) against rigid constraints (portability).

Environment Capture and HPC Stacks

A package list alone is often insufficient.

- Capture Python versions, CUDA runtimes, and resolved dependencies using Conda, lock files, or containers.
- Use Spack for HPC compiler, MPI, and accelerator stacks.
- Document assumptions that tools cannot encode.

Testing and Continuous Integration

Testing reveals changes in expected behavior.

- Unit tests verify components; integration tests verify interactions.
- Small reference datasets enable end-to-end workflow checks.
- CI automates installation and validation checks on changes.

Numerical Validation in Science

Scientific results need tolerance-based comparisons.

- Distinguish floating-point variation from meaningful scientific changes.
- Justify and document numerical tolerances.
- Account for architecture and accelerator differences.

Containers and Portable Execution

Containers package dependencies for reuse.

- Docker supports development and cloud workflows; Apptainer is common in HPC.
- Specify base image, dependencies, and runtime configuration.
- Host kernels and GPU drivers still matter; containers complement documentation.

HPC Reproducibility and Portability

HPC workflows need more than application dependencies.

- Document scheduler scripts and resource allocations.
- Record compiler versions, MPI implementations, GPU types, and thread settings.
- Separate essential requirements from site-specific settings.

Experiment Tracking and Provenance

Connect each reported result to its configuration.

- Record architecture, dataset versions, and random seeds.
- Track hyperparameters and training/evaluation metrics.
- Link outputs to source-code revisions and environment details.

Grounded AI-Assisted Reproducibility

AI assistance must rely on verified evidence.

- Use AI for explanation, prioritization, and troubleshooting.
- Ground outputs in repository files and actual test outcomes.
- Human validation is required: AI-generated claims are not evidence.

Companion Tutorial Series

From best practices to executable learning activities.

- Foundations: evidence-based readiness and environment reconstruction.
- Validation: testing, CI, and scientific validation.
- Advanced: provenance, experiment tracking, and HPC portability.

Hands-on with AI-Assisted Tools

Responsible AI and automated assessment in practice.

- Tutorial 06: Responsible AI-Assisted Reproducibility.
- ReproPilot case study: checker and GUI notebooks.
- Apply transferable principles to grounded repository assessment.

ReproPilot: A Research Prototype

Deterministic evidence combined with grounded AI assistance.

- Eight assessment categories include documentation, dependencies, and HPC stacks.
- Deterministic assessment remains authoritative.
- AI helps explain and prioritize evidence-based gaps.

Lessons from the ReproPilot Benchmark

Artifact presence does not guarantee artifact quality.

- Reported presence–quality correlation: r = 0.407.
- Deterministic tools and AI may prioritize different gaps.
- AI is complementary decision support, not a replacement.

Practical Reproducibility Checklist

Before sharing or publishing scientific software:

- Test installation and execution instructions.
- Declare dependencies and capture environments.
- Include automated tests with meaningful assertions.
- Preserve data/model provenance and specify a software license.

The Future of AI-Assisted Science

Human researchers remain responsible for scientific decisions.

- Explore runtime validation and continuous monitoring.
- Use AI as a careful assistant to evidence-based practice.
- Build sustainable workflows and transferable RSE skills.